SQL QUERY GENERATOR

Naan Mudhalvan Project Report Submitted to **Manonmaniam Sundaranar University** in the Partial fulfilment of the Requirement for the award of the **Degree of Bachelor of Science in Computer Science.**

Submitted By

TEAM ID: SWTID-2026-5198

TEAM MEMBER: A. DOMNIC ABISHA – 23082241802112010

TEAM LEADER: M. ESAKKIAMMAL – 23082241802112011

TEAM MEMBER: N. FATHIMA AFFRA – 23082241802112012

TEAM MEMBER: A.FATHIMA AFILA – 23082241802112013

DEPARTMENT OF COMPUTER SCIENCE

WAVOO WAJEEHA WOMEN'S COLLEGE OF ARTS & SCIENCE

(Accredited by NAAC with 'B++' Grade)

KAYALPATNAM – 628 204

APRIL – 2025

CERTIFICATE

CERTIFICATE

This is to certify that the project work is entitled as **"HTML Quick-Styler" is a bonafide work by**

Submitted By

TEAM ID: SWTID-2026-5198

TEAM MEMBER: A. DOMNIC ABISHA – 23082241802112010

TEAM LEADER: M. ESAKKIAMMAL – 23082241802112011

TEAM MEMBER: N. FATHIMA AFFRA – 23082241802112012

TEAM MEMBER: A.FATHIMA AFILA – 23082241802112013

In partial fulfilment of the requirement for the award of the degree of **BACHELOR OF COMPUTER SCIENCE** from **MANONMANIAM SUNDARNAR UNIVERSITY, TIRUNELVELI,** during the academic year of **2025-2026.**

Signature of the Incharge Staff

Signature of the H.O.D

EXTERNAL EXAMINERS

1.

2.

Place:Kayalpatnam

Date:

DECLARATION

DECLARATION

We hereby declare that the Naan Mudhalvan Project, entitled **" SQL QUERY GENERATOR,"** submitted for the **Bachelor of Science in Computer Science** at **Manonmaniam Sundaranar University** is done. This project was completed by the **Department of Computer Science** at Wavoo Wajeeha Women's College of Arts and Science in Kayalpatnam.

Signature of the Candidate

1.A. DOMNIC ABISHA

2.M. ESAKKIAMMAL

3.N.FATHIMA AFFRA

4.A.FATHIMA AFILA

DATE :

PLACE :Kayalpatnam

CONTENTS

S.NO	TABLE OF CONTENTS	PAGE NO
1	PROJECT DESCRPTION	8
2	SCENARIOS	10
3	A RCHITECTURE OVERVIEW	13
4	PROJECT WORKFLOW	16
5	PROJECT STRUCTURE	21
6	IMPLEMENTATION PHASE	22
7	CONCLUSION	41

PROJECT DESCRIPTION

Project Description

The SQL Query Generator is an intelligent database query assistant designed to simplify the process of writing SQL queries. It utilizes IBM's Granite 3.3 2B Instruct model to convert natural language instructions into accurate and production-ready SQLite SQL queries. This platform addresses the complexity of SQL by enabling users to generate queries using simple English commands instead of manually writing SQL code.

By leveraging the advanced language understanding capabilities of the Granite 3.3 2B model, the system analyzes user input and automatically identifies the relevant database tables and columns. It then constructs appropriate SQL statements, including required WHERE conditions, filters, and logical operations based on the user's request. The system also incorporates safety mechanisms such as keyword filtering and validation to prevent potential SQL injection attacks, ensuring secure query generation.

The application provides a user-friendly experience through an interactive Gradio-based conversational interface, allowing users to communicate with the system in a chat-like environment. In addition to query generation, the platform offers multiple export options, enabling users to save generated queries as PDF reports, text files, or share them through social media platforms.

Overall, the SQL Query Generator enhances database accessibility by combining AI-driven natural language processing, intelligent schema understanding, and security validation. The system allows users to perform complex database operations efficiently without requiring advanced SQL knowledge, making database interaction more intuitive, efficient, and accessible.

SCENARIOS

Scenarios

Scenario 1: User Analytics Query

In this scenario, a business analyst needs to retrieve details of users who registered during the previous month. The system processes the natural language request and understands the time-based context included in the query. It automatically identifies the relevant database table, which is the users table, and selects appropriate columns such as name, email, and signup date. Based on the time reference provided by the user, the system generates a suitable WHERE condition to filter records that fall within the last month. An optimized and secure SQL query is then produced, including an automatic LIMIT clause to manage the number of results returned. The generated query is displayed in a properly formatted SQL code block and is also saved in the query history for future access and export.

Scenario 2: Transaction Filtering

In this use case, a financial analyst requests a list of transaction records where the transaction amount is greater than $1000 and the transactions occurred within the past week. The system analyzes the natural language input and detects the relevant transactions table through keyword identification. It extracts the numerical condition related to the transaction amount and applies a date-based filter representing the previous seven days. Both filtering conditions are logically combined using the AND operator to form an accurate and efficient SQL query. The resulting query is syntactically correct and effectively retrieves the required transaction data.

Scenario 3: Export and Reporting

In this scenario, a team lead intends to distribute the generated SQL queries to project stakeholders for review and documentation. The system provides several export and sharing options to facilitate collaboration. Users can generate professionally formatted PDF reports containing the SQL query, along with additional details such as timestamps and query information. The queries can also be exported as plain text files for documentation or further modification. Furthermore, the system creates ready-to-share links for platforms like WhatsApp and Facebook, allowing users to easily share the generated queries with colleagues. All queries are automatically stored in the query history, ensuring consistent accessibility and standardized formatting across different export formats.

Scenario 4: Security Validation

This scenario highlights the platform's ability to enforce security when potentially harmful input is detected. If a user attempts to include malicious SQL commands in their request, the system immediately identifies unsafe keywords such as DROP, DELETE, or ALTER, as well as common SQL injection patterns like semicolons and comment symbols. Once such patterns are detected, the system blocks the query generation process completely. It then displays a clear error message to inform the user about the security violation. In addition, the system prevents any interaction with the database and records the incident in a security log for auditing and monitoring purposes.

ARCHITECTURE OVERVIEW

Architecture overview

SQL Query Generator is built as a secure, modular platform combining Python backend with IBM Granite 3.3 2B Instruct model for intelligent natural language processing. The architecture prioritizes security, accuracy, and user experience through comprehensive validation, intelligent parsing, and professional export capabilities.

Core Technologies

- Python 3.x: Core programming language for application logic
- IBM Granite 3.3 2B Instruct: AI model for natural language understanding
- Gradio: Interactive chat interface framework
- SQLite: Lightweight relational database for query generation
- ReportLab: PDF generation library for professional reports
- Google Colab: Cloud-based development and deployment platform
- Regular Expressions: Pattern matching for intelligent parsing

Pre-requisites

Software Requirements

- Python 3.8+: Download from python.org
- Google Account: Required for Google Colab access
- Gradio: Install via pip (pip install gradio)
- Transformers: Install via pip (pip install transformers)
- PyTorch: Install via pip (pip install torch)
- ReportLab: Install via pip (pip install reportlab)
- Web Browser: Chrome, Firefox, Safari, or Edge
- Internet Connection: Required for Colab and model access

Knowledge Prerequisites

- Python Basics: Functions, dictionaries, list comprehensions, string manipulation
- SQL Fundamentals: SELECT statements, WHERE clauses, basic query structure
- Gradio Framework: Understanding of chat interfaces and components
- Regular Expressions: Basic pattern matching and text extraction
- Database Concepts: Tables, columns, relationships, data types
- Security: SQL injection prevention, input validation

Hardware Requirements

- Processor: Intel i3 or equivalent (Colab runs on cloud)
- RAM: Minimum 4GB local (Colab provides GPU/TPU)
- Storage: 2GB free space for dependencies and exports
- Display: Minimum 1280x720 resolution for interface

PROJECT

WORKFLOW

Project Workflow

Phase 1: Environment Setup and Model Configuration

This phase focuses on preparing the development environment and configuring the AI model required for SQL query generation.

Activity 1.1: Setting Up the Google Colab Environment

- Create a new notebook in Google Colab for project development.
- Enable GPU runtime to improve model inference performance.
- Install all necessary Python libraries and dependencies required for the system.
- Verify that the environment is properly configured and all packages are functioning correctly.

Activity 1.2: IBM Granite Model Configuration

- Access the Hugging Face repository to obtain the required model.
- Load the Granite 3.3 2B Instruct model into the environment.
- Perform initial tests to confirm the model's inference capability.
- Adjust and configure appropriate generation parameters for accurate query generation.

Phase 2: Core Engine Development

This phase involves developing the core logic responsible for SQL generation and implementing security validation mechanisms.

Activity 2.1: Development of Security Validation System

- Create a predefined list of potentially dangerous SQL keywords.
- Implement detection methods to identify SQL injection patterns.
- Develop an is_safe() validation function to check user inputs.
- Conduct thorough testing to ensure the security filters work effectively.

Activity 2.2: Database Schema Definition

- Define the structure of the users table.
- Define the schema for the orders table.
- Define the schema for the products table.
- Define the schema for the transactions table.
- Implement schema validation logic to ensure queries follow the correct database structure.

Phase 3: Interface Development

This phase focuses on designing and implementing an interactive user interface using Gradio, enabling users to interact with the system through a chat-based environment.

Activity 3.1: Chat Interface Design

- Develop a chatbot component that maintains conversation history.
- Provide a copy button for easily copying generated SQL queries.
- Implement proper message formatting for user and system responses.
- Add SQL syntax highlighting to improve readability.

Activity 3.2: Input Control Implementation

- Create a text input box for natural language queries.
- Add a Generate SQL button to trigger query generation.
- Enable Enter key submission for faster interaction.
- Provide a Clear History button to reset the chat interface.

Activity 3.3: Export Feature Development

- Implement PDF export functionality using the ReportLab library.
- Provide an option to export queries as text files.
- Include timestamps in filenames for better organization.
- Add download buttons for easy file access.

Activity 3.4: Social Media Sharing Integration

- Develop a WhatsApp sharing link generator.
- Implement a Facebook share link generator.
- Format messages properly for social media platforms.
- Use URL encoding to handle special characters in shared links.

Activity 3.5: Custom Interface Styling

- Apply gradient backgrounds for improved visual appeal.
- Style interface buttons using appropriate color schemes.
- Implement a modern dark theme for the interface.
- Ensure the design is responsive and adaptable across devices.

Phase 4: Integration and Testing

This phase ensures that all modules work together seamlessly and that the system performs as expected.

Activity 4.1: System Integration

- Connect all interface components with the backend logic.
- Test the complete query processing pipeline.
- Validate the functionality of the export features.
- Verify proper error handling mechanisms.

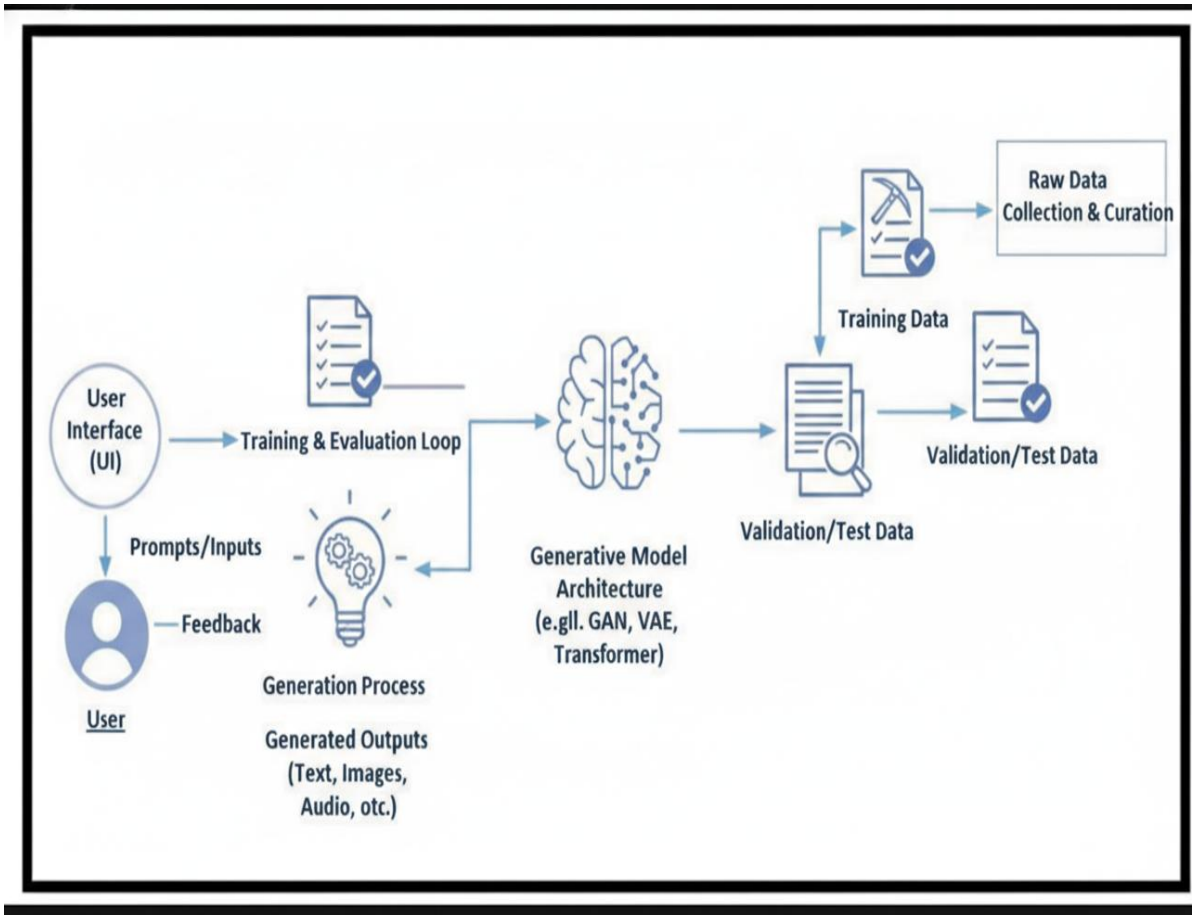
Activity 4.2: Security Testing

- Conduct tests to ensure SQL injection prevention is effective.
- Verify that dangerous SQL keywords are properly blocked.
- Test pattern detection filters thoroughly.
- Confirm that appropriate error messages are displayed when violations occur.

Activity 4.3: Functional Testing

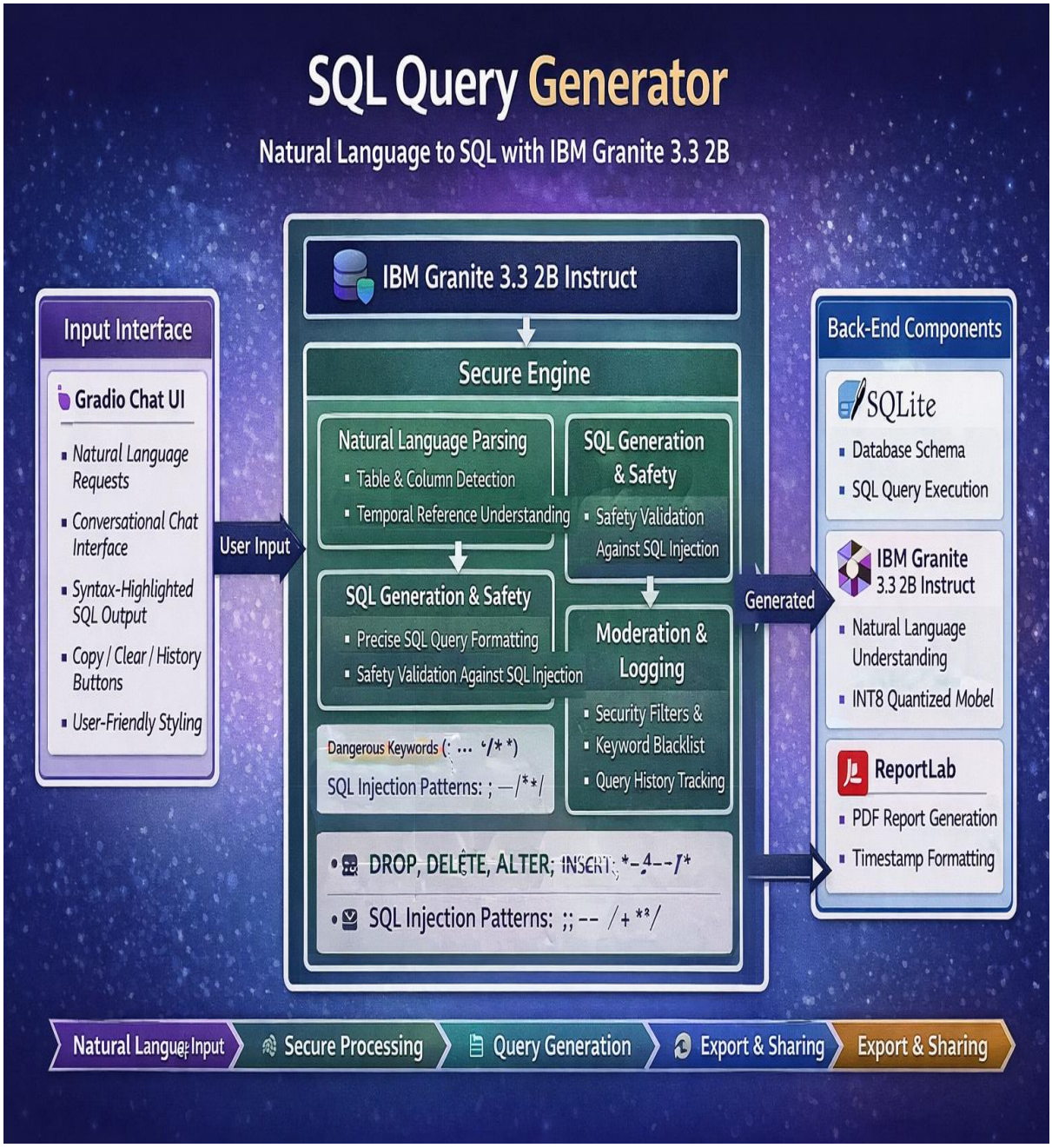
- Test various table detection scenarios.
- Verify the accuracy of column mapping.
- Test extraction of temporal conditions such as date ranges.
- Ensure that all generated queries follow correct SQL syntax.

Technical Architecture



PROJECT STRUCTURE

Project Structure:



IMPLEMENTATION PHASE

Database Schema

Table: users

- id: INTEGER PRIMARY KEY
- name: TEXT
- email: TEXT
- signup_date: DATE
- age: INTEGER
- country: TEXT
- status: TEXT

Table: orders

- id: INTEGER PRIMARY KEY
- user_id: INTEGER (Foreign Key to users.id)
- product_name: TEXT
- amount: REAL
- order_date: DATE
- status: TEXT

Table: products

- id: INTEGER PRIMARY KEY
- name: TEXT
- price: REAL
- category: TEXT
- stock: INTEGER

Table: transactions

- id: INTEGER PRIMARY KEY
- user_id: INTEGER (Foreign Key to users.id)
- amount: REAL
- date: DATE
- type: TEXT

Milestone 1: Environment Setup and Dependency Installation

This milestone focuses on preparing the development environment and installing all required libraries necessary for building the SQL Query Generator system. The implementation is performed using **Google Colab**, which provides a cloud-based Python execution environment with GPU support.

All necessary Python libraries are installed within the Google Colab environment to support the different functionalities of the system, including AI model inference, web interface development, formatting, and report generation.

The following command is used to install the required packages:

```
!pip install -U transformers
```

```
# Use a pipeline as a high-level helper
from transformers import pipeline

pipe = pipeline("text-generation", model="ibm-granite/granite-3.3-2b-instruct")
messages = [
    {"role": "user", "content": "Who are you?"},
]
pipe(messages)
```

```python
# Load model directly
from transformers import AutoTokenizer, AutoModelForCausalLM

tokenizer = AutoTokenizer.from_pretrained("ibm-granite/granite-3.3-2b-instruct")
model = AutoModelForCausalLM.from_pretrained("ibm-granite/granite-3.3-2b-instruct")
messages = [
    {"role": "user", "content": "Who are you?"},
]
inputs = tokenizer.apply_chat_template(
  messages,
  add_generation_prompt=True,
  tokenize=True,
  return_dict=True,
  return_tensors="pt",
).to(model.device)

outputs = model.generate(**inputs, max_new_tokens=40)
print(tokenizer.decode(outputs[0][inputs["input_ids"].shape[-1]:]))
```

```python
!pip install -q gradio transformers torch black autopep8 reportlab requests
```

Activity 1.1: Installing Required Dependencies

Install all required packages in Google Colab:

!pip install -q gradio transformers torch black autopep8 reportlab requests sqlite3

```python
!pip install -q gradio transformers torch black autopep8 reportlab requests sqlite3

import gradio as gr
from datetime import datetime
from pathlib import Path
import json
import sqlite3
import re
from reportlab.lib.pagesizes import letter
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, PageBreak, Table, TableStyle
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.units import inch
from reportlab.lib import colors
import requests
import urllib.parse
```

Package Description

Gradio – Used to build the interactive web interface for the chatbot system.

Transformers – Hugging Face library used to load and run the AI language model.

Torch (PyTorch) – Provides deep learning capabilities required for model inference.

Black – A Python code formatter used to maintain clean and readable code structure.

Autopep8 – Ensures the Python code follows the PEP-8 coding standards.

ReportLab – Library used for generating PDF reports.

Requests – HTTP library used for network operations and integrations.

SQLite3 – Built-in Python module used for database operations.

```python
# ==============================================================================
# DATABASE SCHEMA
# ==============================================================================
SCHEMA = {
    'users': {
        'id': 'INTEGER PRIMARY KEY',
        'name': 'TEXT',
        'email': 'TEXT',
        'signup_date': 'DATE',
        'age': 'INTEGER',
        'country': 'TEXT',
        'status': 'TEXT'
    },
    'orders': {
        'id': 'INTEGER PRIMARY KEY',
        'user_id': 'INTEGER',
        'product_name': 'TEXT',
        'amount': 'REAL',
        'order_date': 'DATE',
        'status': 'TEXT'
    },
    'products': {
        'id': 'INTEGER PRIMARY KEY',
        'name': 'TEXT',
        'price': 'REAL',
        'category': 'TEXT',
        'stock': 'INTEGER'
    },
    'transactions': {
        'id': 'INTEGER PRIMARY KEY',
        'user_id': 'INTEGER',
        'amount': 'REAL',
        'date': 'DATE',
        'type': 'TEXT'
    }
}
```

```python
# ============================================================================
# SECURITY & SQL GENERATION
# ============================================================================
DANGEROUS_KEYWORDS = ['DROP', 'DELETE', 'ALTER', 'TRUNCATE', 'INSERT', 'UPDATE', 'EXEC', 'EXECUTE']
DANGEROUS_CHARS = [';', '--', '/*', '*/', 'xp_', 'sp_']

def is_safe(text):
    """Check if text contains dangerous SQL keywords"""
    text_upper = text.upper()
    for keyword in DANGEROUS_KEYWORDS:
        if keyword in text_upper:
            return False
    for char in DANGEROUS_CHARS:
        if char in text:
            return False
    return True

def get_table_name(query_text):
    """Intelligently infer table name from natural language"""
    query_lower = query_text.lower()

    keyword_map = {
        'users': ['user', 'users', 'customer', 'customers', 'account', 'accounts', 'profile'],
        'orders': ['order', 'orders', 'purchase', 'purchases', 'buy', 'bought'],
        'products': ['product', 'products', 'item', 'items', 'goods'],
        'transactions': ['transaction', 'transactions', 'payment', 'payments', 'transfer']
    }

    for table, keywords in keyword_map.items():
        for keyword in keywords:
            if keyword in query_lower:
                return table

    return 'users'  # Default table
```

Activity 1.2: IBM Granite Model Configuration

In this step, the **IBM Granite 3.3 2B Instruct model** is configured using the Hugging Face model repository. This model provides powerful natural language understanding capabilities that allow the system to interpret user queries and convert them into structured SQL statements.

```python
def get_column_names(table_name, query_text):
    """Map natural language to actual column names"""
    query_lower = query_text.lower()
    columns = list(SCHEMA[table_name].keys())
    selected_cols = []

    column_map = {
        'name': ['name', 'username', 'full name'],
        'email': ['email', 'mail', 'address'],
        'signup_date': ['signup', 'created', 'joined', 'registered', 'date', 'month'],
        'age': ['age', 'years'],
        'amount': ['amount', 'price', 'cost', 'total'],
        'date': ['date', 'when'],
        'status': ['status', 'state'],
        'country': ['country', 'location', 'place']
    }
```

Model Used:

ibm-granite/granite-3.3-2b-instruct

The model is responsible for analyzing natural language inputs, understanding the context of database operations, and assisting in generating accurate SQL queries.

Milestone 2: Security and Database Schema Implementation

This milestone focuses on implementing security mechanisms and defining the database schema structure used by the SQL Query Generator.

Activity 2.1: Security Validation System

A robust security validation mechanism is implemented to prevent **SQL injection attacks** and other malicious query attempts. The system filters user input by detecting dangerous SQL keywords and suspicious query patterns before generating any SQL statement.

Dangerous SQL Keywords Blocked

- DROP
- DELETE
- ALTER
- TRUNCATE
- INSERT
- UPDATE
- EXEC
- EXECUTE

Dangerous Patterns Detected

Semicolons **(;)** – Prevents multiple SQL commands from being chained together.

Comment markers *(--, / /)* – Prevents hiding malicious queries inside comments.

System procedures **(xp_, sp_)** – Blocks attempts to escalate privileges.

If any of these patterns are detected, the system blocks query generation and returns an appropriate error message.

```
for col, keywords in column_map.items():
    if col in columns:
        for keyword in keywords:
            if keyword in query_lower:
                selected_cols.append(col)
                break

if not selected_cols:
    selected_cols = ['*']
else:
    selected_cols = list(dict.fromkeys(selected_cols))  # Remove duplicates

return selected_cols
```

Activity 2.2: Database Schema Definition

The system defines a database schema consisting of four main tables stored within a schema dictionary. These tables simulate a typical business database structure.

Tables Defined in the Schema

Users Table – Stores user account information.

Orders Table – Contains purchase order records.

Products Table – Maintains the product catalog.

Transactions Table – Records financial transaction details.

Each table includes relevant columns with SQLite data types such as **INTEGER, TEXT, REAL, and DATE**.

```python
def extract_conditions(query_text):
    """Extract WHERE conditions from natural language"""
    query_lower = query_text.lower()
    conditions = []

    # Last month
    if 'last month' in query_lower:
        conditions.append("DATE(signup_date) >= DATE('now', '-1 month')")

    # Last year
    if 'last year' in query_lower or 'this year' in query_lower:
        conditions.append("DATE(signup_date) >= DATE('now', '-1 year')")

    # Specific date patterns
    if 'last week' in query_lower:
        conditions.append("DATE(signup_date) >= DATE('now', '-7 days')")

    if 'today' in query_lower:
        conditions.append("DATE(signup_date) = DATE('now')")

    # Active/inactive
    if 'active' in query_lower:
        conditions.append("status = 'active'")
    elif 'inactive' in query_lower:
        conditions.append("status = 'inactive'")

    # Amount filtering
    amount_match = re.search(r'more than|greater than|above\s+([0-9]+)', query_lower)
    if amount_match:
        conditions.append(f"amount > {amount_match.group(1)}")

    amount_match = re.search(r'less than|below\s+([0-9]+)', query_lower)
    if amount_match:
        conditions.append(f"amount < {amount_match.group(1)}")

    return conditions
```

Milestone 3: Natural Language Processing Engine

This milestone focuses on developing the NLP-based engine that interprets user queries and converts them into structured SQL queries.

Activity 3.1: Table Detection System

The system uses a function called **get_table_name()** to identify the appropriate database table based on keywords present in the user's natural language request.

Keyword Mapping for Tables

Users Table Keywords

user, users
customer, customers

account, accounts
profile

Orders Table Keywords

order, orders
purchase, purchases
buy, bought

Products Table Keywords

product, products
item, items
goods

Transactions Table Keywords

transaction, transactions
payment, payments
transfer

This mapping enables the system to automatically determine the relevant table when generating SQL queries.

```python
def generate_sql(query_text):
    """Generate SQLite SQL from natural language"""

    if not is_safe(query_text):
        return None, "❌ UNSAFE: Query contains dangerous SQL keywords!"

    table = get_table_name(query_text)
    columns = get_column_names(table, query_text)
    columns_str = ', '.join(columns) if columns != ['*'] else '*'

    # Build base query
    sql = f"SELECT {columns_str} FROM {table}"

    # Add conditions
    conditions = extract_conditions(query_text)
    if conditions:
        sql += " WHERE " + " AND ".join(conditions)

    # Add limit
    if 'all' not in query_text.lower():
        sql += " LIMIT 10"

    return sql, f"✅ Generated from table '{table}' with columns: {columns_str}"
```

Activity 3.2: Column Mapping System

The **get_column_names()** function maps natural language keywords to database column names.

Column Keyword Mapping

name → name, username, full name

email → email, mail, address

signup_date → signup, created, joined, registered, date, month

age → age, years

amount → amount, price, cost, total

status → status, state

country → country, location, place

This process ensures that user requests referencing attributes in plain English are accurately mapped to the correct database columns.

```
def process_query(user_query, chat_history):
    """Process database query request"""

    if not user_query.strip():
        return chat_history

    chat_history.append((user_query, "Processing..."))

    sql_query, explanation = generate_sql(user_query)

    if sql_query is None:
        response = f"❌ ERROR\n\n{explanation}"
    else:
        response = f"✅ SQL GENERATED\n\n```sql\n{sql_query}\n```\n\n📝 {explanation}"
        query_history.append({
            'user': user_query,
            'sql': sql_query,
            'timestamp': datetime.now().isoformat()
        })

    chat_history[-1] = (user_query, response)
    return chat_history
```

Activity 3.3: Condition Extraction

The **extract_conditions()** function analyzes user queries to detect filtering conditions. It uses **regular expressions (regex)** and keyword detection to identify numerical filters, date ranges, and logical conditions.

For example:

“greater than 500”

“last week”

“last month”

These phrases are automatically converted into appropriate SQL **WHERE conditions**.

```
def clear_chat():
    global query_history
    query_history = []
    return []
```

Activity 3.4: Social Sharing Feature Development

The system includes features that allow users to share generated SQL queries through social media platforms. Shareable links are generated for **WhatsApp and Facebook**, enabling quick collaboration among team members.

```
def download_pdf():
    if not query_history:
        return "❌ No queries to export!", None

    timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
    filename = f"sql_queries_{timestamp}.pdf"

    try:
        doc = SimpleDocTemplate(filename, pagesize=letter)
        styles = getSampleStyleSheet()
        story = []

        title_style = ParagraphStyle(
            'CustomTitle',
            parent=styles['Heading1'],
            fontSize=26,
            textColor='#FF6B6B',
            spaceAfter=30,
            alignment=1
        )

        story.append(Paragraph("🔍 SQL Query Report", title_style))
        story.append(Paragraph(f"Generated: {datetime.now()}", styles['Normal']))
        story.append(Spacer(1, 0.3*inch))

        for i, query in enumerate(query_history, 1):
            story.append(Paragraph(f"<b>Query #{i}</b>", styles['Heading2']))
            story.append(Paragraph(f"<b>User Request:</b> {query['user']}", styles['Normal']))
            story.append(Spacer(1, 0.1*inch))
            story.append(Paragraph(f"<b>Generated SQL:</b>", styles['Normal']))
            story.append(Paragraph(f"<font color='#4ECDC4'><code>{query['sql']}</code></font>", styles['Normal']))
            story.append(Spacer(1, 0.2*inch))

        doc.build(story)
        return f"✅ PDF Downloaded: {filename}", filename
    except Exception as e:
        return f"❌ Error: {str(e)}", None
```

Activity 3.5: Custom Interface Styling

To improve usability and visual appeal, the interface includes several styling enhancements such as:

Gradient backgrounds

Modern dark theme

Styled buttons with consistent color schemes

Responsive layout design

These features create a modern and user-friendly application interface.

```
def download_txt():
    if not query_history:
        return "❌ No queries to export!", None

    content = f"SQL Query History Report\nGenerated: {datetime.now()}\n{'='*60}\n\n"

    for i, query in enumerate(query_history, 1):
        content += f"Query #{i}\n"
        content += f"User Request: {query['user']}\n"
        content += f"Generated SQL: {query['sql']}\n"
        content += f"Timestamp: {query['timestamp']}\n"
        content += f"{'-'*60}\n\n"

    timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
    filename = f"sql_queries_{timestamp}.txt"

    try:
        Path(filename).write_text(content)
        return f"✅ TXT Downloaded: {filename}", filename
    except Exception as e:
        return f"❌ Error: {str(e)}", None
```

Milestone 4: Gradio Interface and Export Features

This milestone focuses on creating the interactive user interface and implementing export functionality.

Activity 4.1: Chat Interface Implementation

The system uses **Gradio** to build a conversational interface where users can interact with the SQL Query Generator.

Interface Components

Chatbot component that stores conversation history

Natural language input text box

Generate SQL button for query creation

Copy button for easily copying generated SQL

Clear history button for resetting the conversation

Custom CSS styling with gradient themes

```python
def generate_whatsapp_link():
    if not query_history:
        return "❌ No queries to share!"

    message = "🔍 *SQL Queries Generated*\n\n"
    for i, query in enumerate(query_history[-5:], 1):
        message += f"*Query {i}:* {query['user']}\n"
        message += f"`{query['sql']}`\n\n"

    encoded_message = urllib.parse.quote(message)
    whatsapp_link = f"https://wa.me/?text={encoded_message}"

    return f"✅ WhatsApp Link Created!\n\n🔗 {whatsapp_link}\n\n📱 Click to open WhatsApp and share!"
```

Activity 4.2: PDF Export System

The platform includes a **PDF export feature** implemented using the ReportLab library. This allows users to generate professional query reports.

Features of the PDF Report

Professional report title

Query generation timestamp

Numbered query sections

Display of user request

Generated SQL query

Automatically generated filename with timestamp

```python
def generate_facebook_link():
    if not query_history:
        return "❌ No queries to share!"

    message = "Check out these SQL queries I generated: "
    for query in query_history[-3:]:
        message += f"{query['user']} → {query['sql']} | "

    encoded_message = urllib.parse.quote(message)
    facebook_link = f"https://www.facebook.com/sharer/sharer.php?quote={encoded_message}"

    return f"✅ Facebook Link Created!\n\n🔗 {facebook_link}\n\n📱 Click to share on Facebook!"

print("🚀 Loading SQL Query Generator...")
```

Activity 4.3: Social Sharing Integration

The system integrates social sharing features that allow users to share generated SQL queries easily.

Sharing Features

WhatsApp share link with formatted message

Facebook share link for posting queries

URL encoding for handling special characters

Last five queries included in WhatsApp share

Last three queries included in Facebook share

Direct click-to-share functionality

```
with gr.Blocks(
    title="SQL Query Generator",
    theme=gr.themes.Base(),
    css="""
    body { background: linear-gradient(135deg, #1a1a2e 0%, #16213e 100%); }
    .gradio-container { background: linear-gradient(135deg, #1a1a2e 0%, #16213e 100%); }
    .chatbot-wrapper { background: #0f3460; border-radius: 15px; }
    .button-primary { background: linear-gradient(135deg, #FF6B6B 0%, #FF8E72 100%); }
    .button-secondary { background: linear-gradient(135deg, #4ECDC4 0%, #44A08D 100%); }
    .button-danger { background: linear-gradient(135deg, #FF6B6B 0%, #EE5A6F 100%); }
    """
) as demo:

    gr.Markdown("""
    # 🔍 SQL Query Generator
    ## Natural Language → SQLite SQL

    Convert plain English requests into safe, production-ready SQL queries!
    """)

    with gr.Row():
        with gr.Column(scale=3):
            chatbot = gr.Chatbot(
                label="💬 Query Conversion",
                height=450,
                show_copy_button=True,
                bubble_full_width=False
            )

            with gr.Row():
                query_input = gr.Textbox(
                    label="📝 Natural Language Request",
                    placeholder="e.g., Show me all users who signed up last month",
                    lines=2,
                    scale=4
                )
                send_btn = gr.Button("Generate SQL", scale=1, size="lg", variant="primary")

            send_btn.click(process_query, inputs=[query_input, chatbot], outputs=[chatbot])
            query_input.submit(process_query, inputs=[query_input, chatbot], outputs=[chatbot])

        with gr.Column(scale=1):
            gr.Markdown("""
            ### 🎮 Controls
            """)
```

Milestone 5: Testing and Deployment

The final milestone focuses on deploying the system and validating its functionality through testing.

Activity 5.1: Deployment Using Google Colab

The application is deployed using **Gradio's sharing feature**, which generates a publicly accessible link.

Launch Configuration

share=True – Generates a public shareable link

show_error=True – Displays detailed error messages

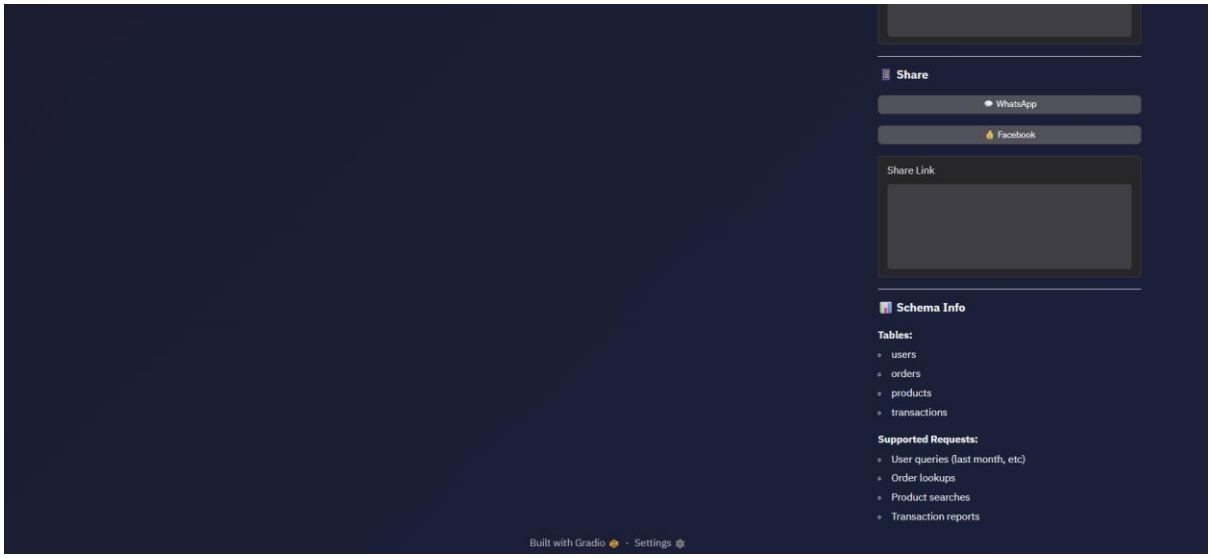
show_api=False – Hides API documentation from users

Once deployed, the application can be accessed through a public URL generated by Gradio.

```
        clear_btn = gr.Button("🗑️ Clear History", variant="stop", size="sm")
        clear_btn.click(clear_chat, outputs=[chatbot])

        gr.Markdown("---")
        gr.Markdown("### 📥 Export")

        pdf_btn = gr.Button("📄 Download PDF", size="sm", variant="primary")
        txt_btn = gr.Button("📃 Download TXT", size="sm", variant="primary")

        pdf_file = gr.File(label="PDF File", visible=True)
        txt_file = gr.File(label="TXT File", visible=True)

        export_status = gr.Textbox(label="Status", interactive=False, lines=2)

        pdf_btn.click(download_pdf, outputs=[export_status, pdf_file])
        txt_btn.click(download_txt, outputs=[export_status, txt_file])

        gr.Markdown("---")
        gr.Markdown("### 🔗 Share")

        whatsapp_btn = gr.Button("💬 WhatsApp", size="sm", variant="secondary")
        facebook_btn = gr.Button("👍 Facebook", size="sm", variant="secondary")

        share_output = gr.Textbox(label="Share Link", interactive=False, lines=5)

        whatsapp_btn.click(generate_whatsapp_link, outputs=[share_output])
        facebook_btn.click(generate_facebook_link, outputs=[share_output])

        gr.Markdown("---")
        gr.Markdown("""
        ### 📊 Schema Info

        **Tables:**
        - users
        - orders
        - products
        - transactions

        **Supported Requests:**
        - User queries (last month, etc)
        - Order lookups
        - Product searches
        - Transaction reports
        """)

print("\n✅ SQL Query Generator Ready!\n")
demo.launch(share=True, show_error=True, show_api=False)
```

Access Points:

- Public URL: https://e35025e6ba7e5cb298.gradio.live/



Activity 5.2: Functional Testing

Several test cases are used to validate the system's performance and accuracy.

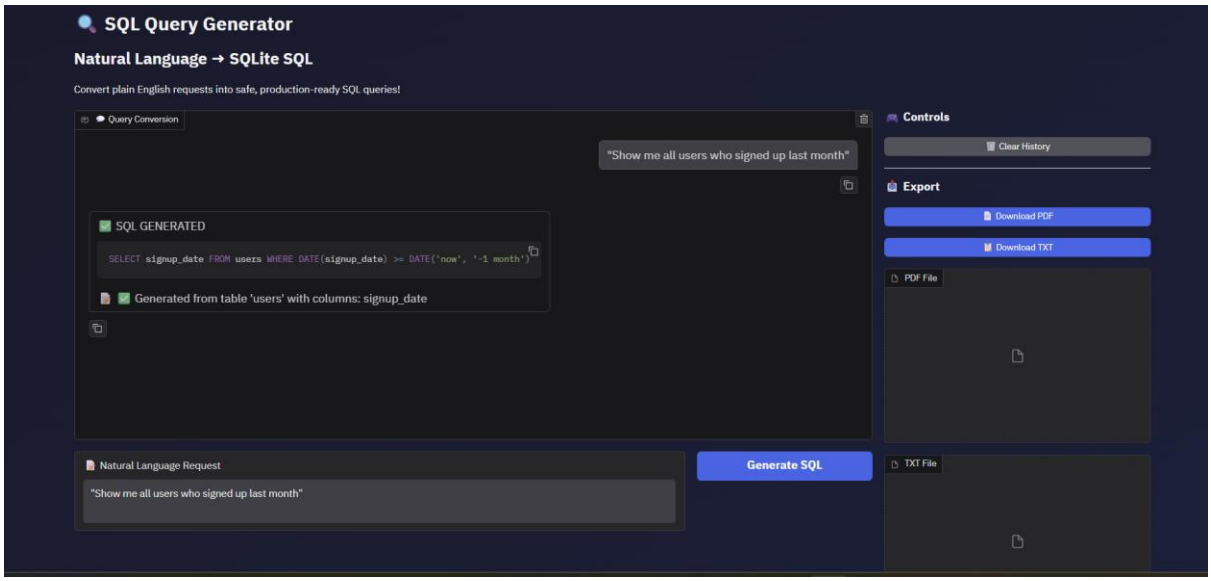
Test Case 1: User Signup Query

Input

"Show me all users who signed up last month"

Expected SQL Output

SELECT * FROM users
WHERE DATE(signup_date) >= DATE('now','-1 month')
LIMIT 10;



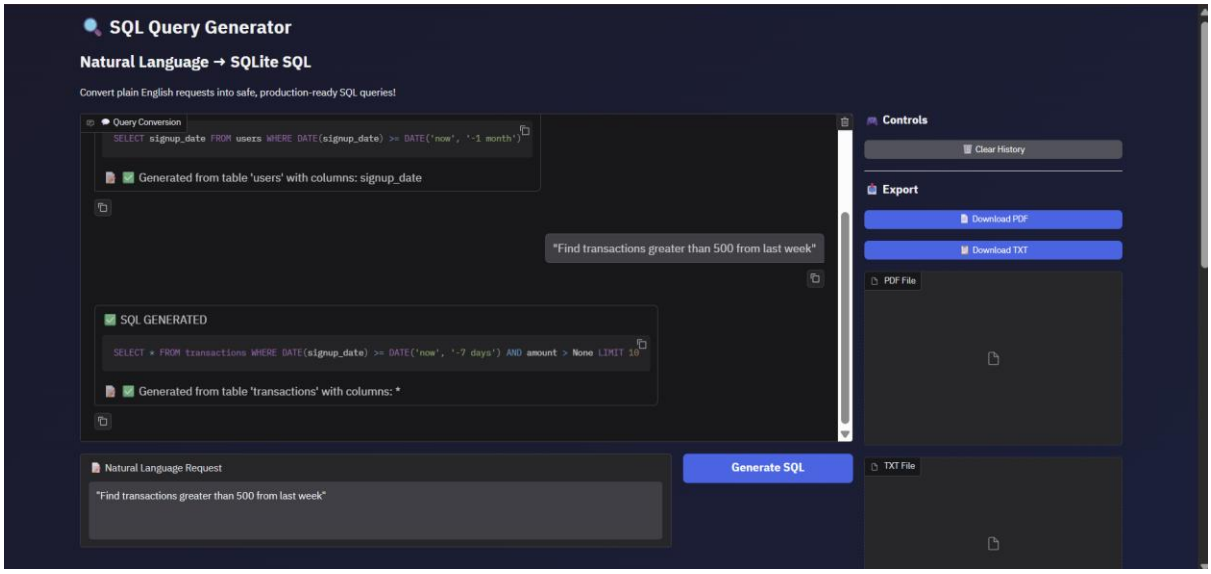
Test Case 2: Transaction Query with Amount Filter

Input

"Find transactions greater than 500 from last week"

Expected SQL Output

```
SELECT * FROM transactions
WHERE amount > 500
AND DATE(date) >= DATE('now','-7 days')
LIMIT 10;
```



Conclusion:

The SQL Query Generator successfully provides an intelligent database query assistant that converts natural language requests into structured SQL queries. By integrating the IBM Granite 3.3 2B Instruct model with advanced natural language processing techniques, the system enables users to generate accurate SQL queries without requiring deep knowledge of SQL syntax.

The platform also incorporates a strong security validation system to prevent SQL injection attacks and malicious inputs. Additionally, the application offers professional export features including PDF reports, text file downloads, and social media sharing, making it suitable for both individual analysts and collaborative teams.

Through its Gradio-based conversational interface, intelligent query interpretation, and comprehensive feature set, the SQL Query Generator transforms database interaction into a simple and user-friendly process.